

PAR Laboratory Assignment

Lab 5: Geometric data decomposition implementation and
analysis

Mario Acosta, Eduard Ayguadé, Rosa M. Badia, Jesús Labarta, Pau Ferrer,
Daniel Jiménez-González, Pablo Vizcaino, and Gladys Utrera

Spring 2025–2026



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Departament d'Arquitectura de Computadors

Contents

1	Experimental Setup	5
1.1	Laboratory files	5
1.2	Mandelbrot parameters	6
1.3	Compilation	6
1.4	Execution	6
1.5	Analysis	6
2	Geometric data decomposition strategies	7
2.1	Implementation	7
2.2	Execution	7
2.3	Correctness	8
2.4	Performance Analysis	8
A	Laboratory deliverable information	9
A.1	Evaluation of English as a third language	10
A.2	Submission of the deliverable and source codes to Atenea	10

Important!

We have simplified the sequential mandel source code. However, the data dependency analysis that you did in lab3 is still valid. The new version is now more simple and there are not tiles neither checking of the border tiles. The computation is always done for the full matrix. Your main objective is to analyse three different geometric data decompositions and compare them each other.

Annex **A** describes the laboratory document that you will need to elaborate and upload to Atenea together with all your C source codes.

Chapter 1

Experimental Setup

1.1 Laboratory files

This laboratory assignment will be done in two sessions of two hours each. All the files necessary to carry out this laboratory are available in a compressed file at `/scratch/nas/1/par0/sessions/lab5.tar.gz` located in `boada.ac.upc.edu`.

Uncompress the aforementioned file in your home directory with these command lines:

```
> cd
> tar -zxvf ~par0/sessions/lab5.tar.gz
```

You will find the following files in the `lab5` directory:

- `Makefile`.

- Source codes:

mandel-seq-iter-simple.c: Simple (no tiled) iterative sequential code.

mandel-omp-iter-simple.c: Incomplete simple (no tiled) iterative baseline OpenMP code.
You must implement additional OpenMP codes using this baseline file.

- Scripts:

submit-seq.sh: Script to execute the sequential code and useful to obtain the expected outputs of Mandelbrot. You should use the outputs of the iterative sequential code to check the correctness of your parallel programs.

submit-omp.sh: Script to execute an OpenMP parallel code and useful to check the correctness of your programs and find out the performance of your parallel program for a particular number of threads.

submit-strong-omp.sh: Script to perform the strong scalability of OpenMP parallel codes that generates two output files with the information of the speedup and elapsed time of the Mandelbrot function:

speedup-partial-program-hostname.txt: Speedup file.

elapsed-partial-program-hostname.txt: Elapsed execution times file used to fill in Table [A.2](#).

submit-strong-extrae.sh: Script to perform the Modelfactors analysis of OpenMP parallel codes. Remember that its execution also generates a set of Paraver traces within the Modelfactors directory.

submit-strong-memory.sh: Script to perform the memory hierarchy analysis of an OpenMP parallel code. Its execution generates a set of Paraver traces as well as a text file with the information of the number of L2 cache misses used to fill in Table [A.2](#).

1.2 Mandelbrot parameters

The arguments to be pass to the executables can be seen by running the command `mandel -help`:

```
./mandel-seq-iter-simple -help
Usage: ./mandel-seq-iter-simple [-o -h -d -i maxiter -c x0 y0 -s size]
  -o to write computed image (mandel_image.jpg) and histogram \
      (mandel_histogram.out if -h indicated) to disk (default no file generated)
  -h to produce histogram of values in computed image (default no histogram)
  -d to display computed image (default no display)
  -i to specify maximum number of iterations at each point (default 100)
  -c to specify the center x0+iy0 of the square to compute
  -s to specify the size of the square to compute (default 2, i.e. size 4 by 4)
```

Warning!

The `-o` option makes the program to write the image and histogram to disk always using with the *exact same* name. Therefore, rename them if you want to keep them and later compare them with your future parallel outputs. Both of these files are stored in binary format so that you will need to use `cmp` or `diff` commands to compare them.

1.3 Compilation

Look at the `Makefile` to understand how to compile each executable.

1.4 Execution

We have provided you with a set of scripts to analyse your parallel implementations.

1.5 Analysis

We suggest you to review the Lab 1 sections and annexes to remember the functionality of `Modelfactors` and how to use it to generate and analyse your OpenMP implementations. Remember also to look at your Lab 3 deliverable to remember everything you did.

Chapter 2

Geometric data decomposition strategies

2.1 Implementation

Important!

You are not allowed to use the `for` worksharing clause of OpenMP.

We ask you to implement the following decomposition strategies:

- 1D block geometric data decomposition by columns.** An iterative data decomposition that should minimize the **data** load unbalance due to this data decomposition and exploit data locality accessing matrix **M** as much as possible.
- 1D block-cyclic geometric data decomposition by columns.** An iterative data decomposition that should minimize the **data** and **computational** load unbalance due to this data decomposition. Important: you should keep in mind to reduce cache coherence protocol overheads and exploit data locality accessing matrix **M** as much as possible.
- 1D cyclic geometric data decomposition by rows.** An iterative data decomposition that should minimize the **data** and **computational** load unbalance due to this data decomposition, reduce cache coherence protocol overheads, and exploit data locality accessing matrix **M** as much as possible.

2.2 Execution

Run the sequential code and the three parallel implementations to check whether they work:

1. Run the sequential code with:

```
sbatch submit-seq.sh mandel-seq-iter-simple -h -o -i 256
```

This will generate a histogram (`mandel_histogram.out`) and an image (`mandel_image.jpg`) as outputs.

2. Rename the output of the sequential code.
3. Run each parallel implementation with 20 threads to generate a histogram (`mandel_histogram.out`) and an image (`mandel_image.jpg`) as outputs:

```
sbatch submit-omp.sh mandel-omp-iter-simple 20
```

2.3 Correctness

Compare the two output files:

- the histogram `mandel_histogram.out`,
- the image `mandel_image.jpg`,

between the sequential and parallel versions. You can use `cmp` or `diff` Linux commands. Should these files differ, you will have to review your code and modify it accordingly.

2.4 Performance Analysis

We ask you to run the Modelfactors analysis, analyse the Paraver trace with 16 threads, and perform a strong scaling analysis by performing the following steps:

1. Execute the Modelfactors analysis:

```
sbatch submit-strong-extrae.sh mandel-omp-iter-simple
```

Look at the Lab 1 document to remember how to read and understand the Modelfactors tables.

2. Perform a memory hierarchy analysis:

```
sbatch submit-strong-memory.sh mandel-omp-iter-simple
```

It will generate execution traces and two text files with the overall number of L2 cache misses.

3. Open the execution trace corresponding to 16 threads and load the `l2d_implicit.cfg` configuration file on Paraver to show the amount of L2 cache misses per thread.
4. Perform a strong scalability analysis of the parallel code:

```
sbatch submit-strong-omp.sh
```

We recommend you to take into account all the analysis done in Lab 3.

Appendix A

Laboratory deliverable information

Table A.1 explains what should be included in the report.

Table A.1: Analysis to be included in the PDF deliverable report and codes to be added into the zip file.

Section	Description
Geometric decomposition	Name of the geometric decomposition.
Code	Add the parallel source code of each implementation (Block Geometric Data Decomposition by columns, Block-Cyclic Geometric Data Decomposition by columns and Cyclic Geometric Data Decomposition by rows) to the compressed file. Indicate the name of the source code in the PDF. Please refrain from including the entire code in the document, except for fragments of codes that you consider necessary to explain your work.
Modelfactor analysis	Include the Modelfactors tables to the report for each implementation and compare the results for the three versions. Remember that the Modelfactors analysis generates a directory with a set of execution traces and a summary file: <code>modelfactors.out</code> or <code>modelfactor-tables.pdf</code> .
Paraver analysis	Include the Paraver views of the execution trace with 16 threads using the following hints: <code>Useful/Instantaneous parallelism</code> and <code>OpenMP/Implicit tasks in parallel constructs</code> of the three implementations. Please make sure that all the traces have the same duration and size (i.e., adjust for the longest execution trace) in order to facilitate the comparison between the three implementations.
Memory analysis	For each parallel code you should submit the <code>submit-strong-memory.sh</code> script. This will generate several traces in a directory called <code>programName-strong-memory</code> and a file with the name <code>l2_misses_...txt</code> . These files contain the overall number of cache misses (the total accumulated by all threads) as well as the average number of misses per thread corresponding to the second level of cache memory. You must include this information in your report. For a specific number of threads (e.g., 16), open the corresponding Paraver trace, use the configuration file <code>l2d_implicit.cfg</code> that you will find in the <code>lab5</code> directory, and include the Paraver in the deliverable. Compare the three implementations based on the number of L2 cache misses and try to reason the differences between them.
Strong scalability	For each data decomposition parallel code, you should include the scalability plot to the report and compare the three versions.

Final comparison between all the parallel strategies

Table A.2 should be included after the description of all the parallel strategies. Note that you have to fill in this table using the output files generated after the execution of the `submit-strong-memory.sh` and `submit-strong-omp.sh` scripts. Based on your performance results, reason which implementation you consider is the best.

Table A.2: Summary of the elapsed execution times for each version, obtained from the output files after the execution of `submit-strong-omp.sh` script and average number of L2 cache misses per thread obtained from the output files generated by the `submit-strong-memory.sh` script.

Version	Number of threads					
	1	4	8	16	24	32
1D Block geometric data decomposition by columns						
1D Block-cyclic geometric data decomposition by columns						
1D Cyclic geometric data decomposition by rows						
Version	Number of threads (L2 cache misses per thread)					
1D Block geometric data decomposition by columns						
1D Block-cyclic geometric data decomposition by columns						
1D Cyclic geometric data decomposition by rows						
Best parallel implementation	Version	Reason why				

A.1 Evaluation of English as a third language

As you should know, this course contributes to the transversal competence “Tercera llengua”. Please deliver your material in English if you want this competence to be evaluated and also refer to the “Rubrics for the third language competence evaluation” document to know which rubrics will be utilised.

A.2 Submission of the deliverable and source codes to Atenea

An entry for the submission will be created in Atenea and the proper submission deadlines will be set. Only a PDF format for this document will be accepted for submission. Additionally, you are required to submit complete source codes with the Tareador instrumentation that you have developed. Please refrain from including the entire source code in the deliverable document, except for fragments of codes that you consider necessary to explain your work.

Important!

You must upload two separate files to Atenea:

1. The deliverable document in PDF format.
2. A compressed file (zip) containing all the requested source codes.